Piazza API

About

A RESTful social media SaaS called Piazza. In Piazza, users post messages for a particular topic while others browse posts and perform basic interactions, including liking, disliking, or adding a comment. This is a courswork project for the Cloud Computing module at Birkbeck, University of London.

Getting started

Install node and yarn or npm on your computer. Create a .env file and populate it with the database connector string that you can retrieve from mongodb.

Setup

Development Setup

The project uses `yarn` as package manager, `prettier` as formatter, `eslint` as linter and `nodemon` to run the development server and track changes during development. These are all listed as 'devdependencies' in the package.json file. I am used to have these helpers in my projects and combined with the VSCode settings, they help me to focus on the logic of the code.

Project Setup

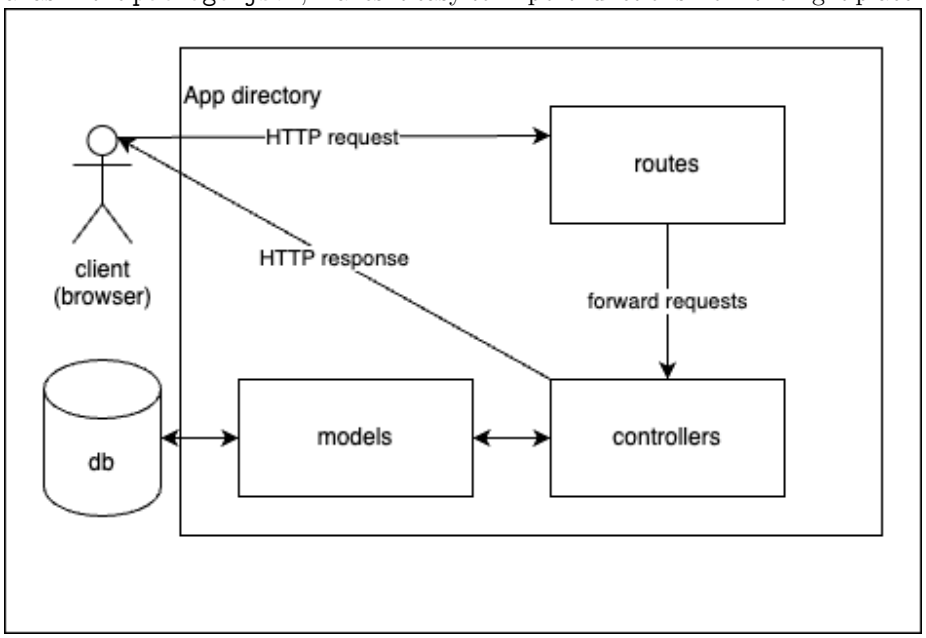
The Api is a Node.js application that uses the Express framework. The docker container that will be deployed to the virtual server in the cloud will have Node installed. This means Node will run the app on the virual server. During development, however, Nodemon is used, hence it is a developent dependency. I set the the eslint config on purpose so that I can use modules, as we use a modern node version and all the dependencies used in the project are compatible.

Phase A

App Directory Structure

All the logic is in the `App` directory. This helps to create a more maintainable and scalable structure for the app. As suggested in this article this article (Folder Structure for NodeJS & ExpressJS project, 2022) the directory is called `App` and instead of `Src` because the files run on the server and will not be compiled, transpiled, and minified to be sent to the client. The routes forward the request to their corresponding controllers, which will then send the response. This is a good practise shown in the Mozilla Express Tutorial (Express Tutorial Part 4: Routes and controllers - Learn web development | MDN, 2024) Adding the `App` directory as 'imports'

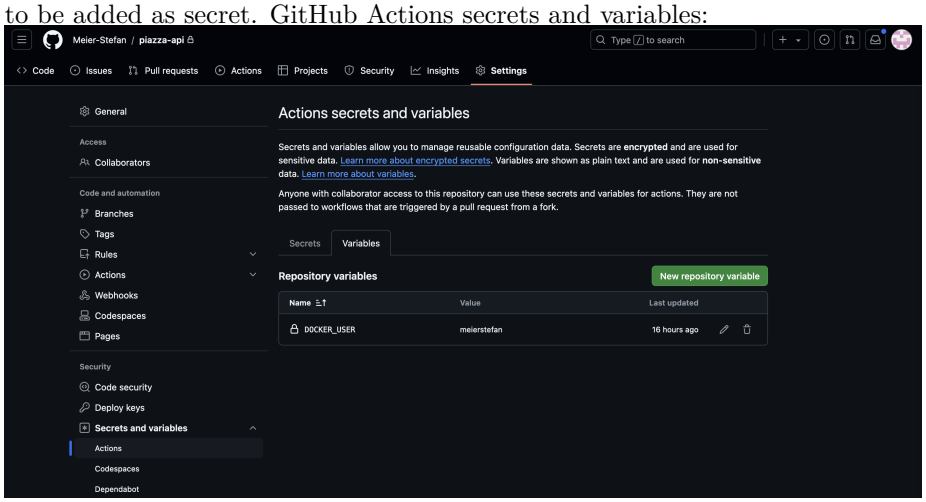
alias in the `package.json`, makes it easy to import functions from the right place.



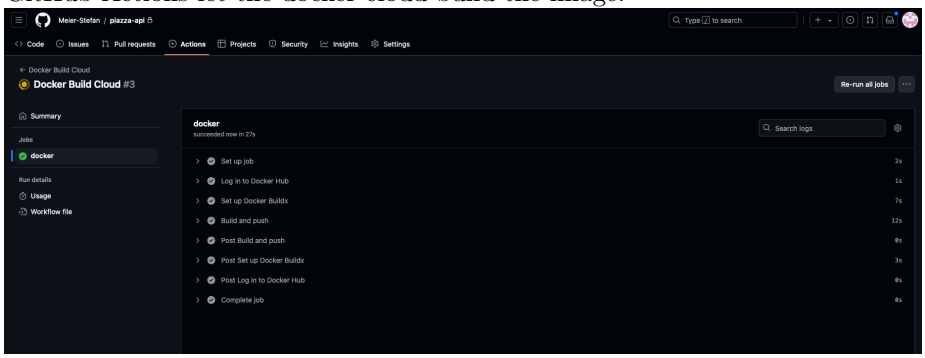
Infrastructure Setup Description

The app is containerized as described in the docker docs the docker docs (Part 1: Containerize an application, 100AD). The only thing that needs to be adjusted is on line 7: the app.js sits in the root directory and is not called index.js.

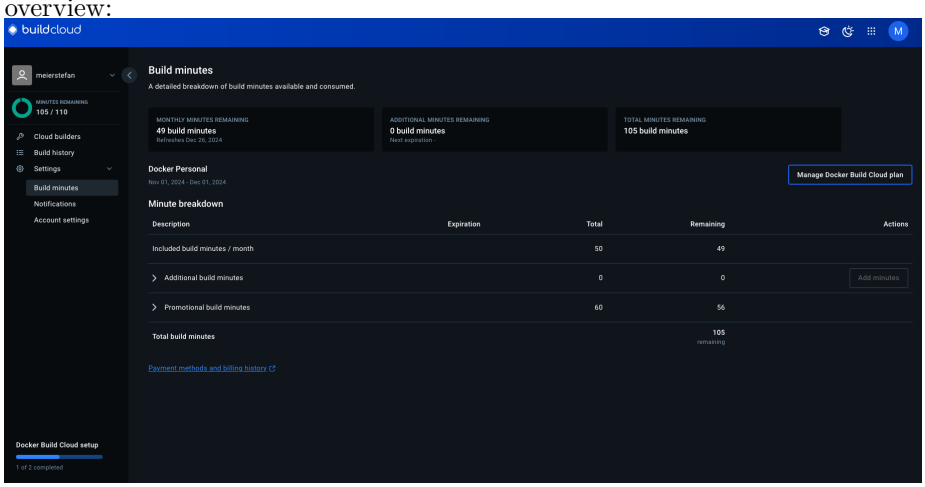
Docker Build Cloud is used in CI (continuous integration) (Continuous integration, 100AD). For this, the docker username must be added as environment variable in the GitHub Repository. And a Personal Access Token (PAT) needs to be added as secret. GitHub Actions secrets and variables:



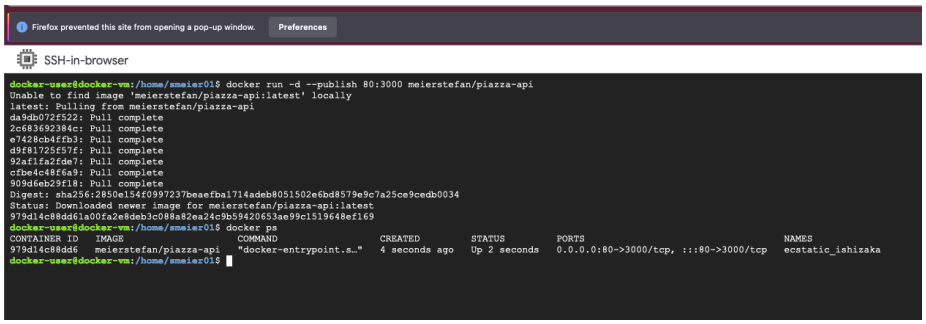
The docker manual triggers on push to the main branch, but I decided to use the manual workflow_dispatch event trigger instead. In the docker build cloud, a cloud builder needs to be added, I added one and called it `piazza-builder`. GitHub Actions let the docker cloud build the image:



Docker Build Cloud is fast, but it costs to build. They gave me 60 free minutes because I implemented CI from Github Actions, thanks Docker. Build minutes overview:



As a result, the image was available on Docker Hub: Docker Hub Images
To make the REST API endpoints available in the virtual machine, Docker was installed on the virtual machine and the docker-user was created like in lab 5.1 (cc/Class-5/Lab5.1 Introduction to Docker.md at master · warestack/cc, no date). Video of lab 5.1 (A Docker tutorial, 2022). The docker-user was logged in on docker hub using the PAT method from above. Then the docker container was started with port mapping like shown in lab 5.1. Because the image was only in the docker cloud, it was automatically pulled before the container was started:



Once the container was up and running, the endpoints were available under the virtual machine IP address. The following screenshots show the Home page, post list endpoint, topic detail, topic list and user profile endpoints:

Hello World!

Post list endpoint

Topic detail endpoint

Topic list endpoint

User profile endpoint

Phase B

Connecting to the database

To start the development server, the node environment is specified in the 'dev' script in the package.json file. So the app knows from which .env dotfile to take the environment variables. The main function currently only connects to the database using the `connect` function from mongoose and console logs when it started and if it worked.

Register a user

When a user posts to the `user/register` endpoint, route will forward to the corresponding controller. That controller uses a validation function to make sure that the input makes sense and is secure. The validation function is built with the `string, required, min, max` and `email` functions from the `joi` library.

> **IDEA:** For this purpose, the functions were used in a simple way, but the joi api reference (joi.dev, no date) documents ways how this could be improved.

Using the `bcryptjs` library, the password is hashed and 5 rounds of salt are used before it gets stored in the database.

Login as user

When a user posts to the `user/login` endpoint, there is, additional to the validation with `joi`, the password gets compared with a function from `bycryptjs` and, if the password matches, the response contains a json web token, that can later be used to access resources that require authentication.

The `jsonwebtoken` library creates that token using a secret variable from the .env dotfile.

> **SPECIAL:** The json web token expires after one hour and contains the user ID as payload (jsonwebtoken, 2023). This user Id can for example be used by the controllers to make sure that one user does not modify the profile of another user.
>
> **IDEA:** It would be useful if the token would refresh.

Interact with an endpoint that requires authentication

To keep the repository clean and organszed, a middleware folder is used like suggested by this tutorial (Node.js Authentication and Authorization with JWT: Building a Secure Web Application, 2023). The routes that require use the `isAuthenticated` middleware function to make sure that the user is logged in and allowed to proceed. Logged in means that the request headers contain a key value pair of a key containing `auth-token` and a value containing the json web token.If the user is not logged in, or has a wrong json web token, `isAuthenticated` will send the corresponding response instead of letting the route forwarding the request to the controller.

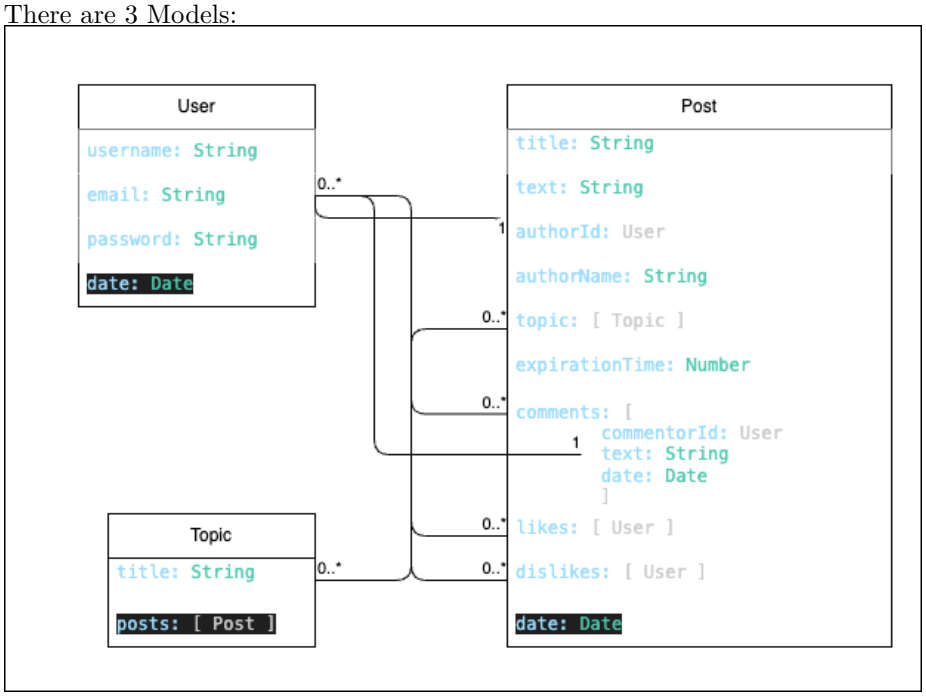
Phase C

Models

The Models are created so that they are practical to use in this small project.

As Joe Karlsson from mongodb explains in this video (MongoDB Schema Design Best Practices, 2021), it is a good idea to think about what will be displayed when designing the database schema. I tied to follow this advice, but also, I did try not to over optimise as if this app was serving millions of users already.

There are 3 Models:



The above diagram follows the style of the mozilla tutorial (Express Tutorial Part 3: Using a Database (with Mongoose) - Learn web development | MDN, 2024) that also uses mongoose. `Users` can create zero or many `Posts`, `Comments`,

Likes, and Dislikes. Each Post and each Comment have one User as author. Likes and Dislikes are arrays that can have zero or many Users listed. Each Post must have at least one Topic, but a Topic has zero or many Posts.

Post


```
_id: ObjectId('67559acabca568c1d9249ece')
title : "Best invention"
text : "What is the best technology invented? Leave it in the comments below."
authorId : ObjectId('674c2c2d9e0f87ce0fb9bd84')
authorName : "stefan"
▸ topic : Array (1)
expirationTime : 45
▸ likes : Array (2)
▸ dislikes : Array (empty)
▾ comments : Array (1)
  ▾ 0: Object
      commentorId : ObjectId('674c2ea85ad0fadd9b2381c8')
      text : "Windows XP"
      _id : ObjectId('67559b50bca568c1d9249ede')
      date : 2024-12-08T13:12:48.872+00:00
date : 2024-12-08T13:10:34.625+00:00
__v : 0
```


The comments are and array of documents in the post documents for now. This is how it will be displayed.

Reactions and comments conatain the link to the user like this: `likes: [{ type: Schema.Types.ObjectId, ref: "users" }],`

> **IDEA:** For projects that could have milions of users, it would make sense to store the comments in separate collections and also to track likes in documents that store which user liked/disliked which post. Both of the above will help to avoid problems with the 16mb document limit and improve the query speed.

Topic


```
_id: ObjectId('674c7965f3a45b229d3b02d9')
title : "Health"
▾ posts : Array (1)
    0: ObjectId('67559f7cfd6bf5c729198a52')
```


The topic is simple, it just contains a title and an array of posts. However, currently I don't use that array. If I display the posts of a topic, I use the post list controller and filter fo the topic id.

> **IDEA:** The likes and dislikes are arrays of userIds. This can be displayed as number in the frontend. The user can then click/hover on these reaction numbers to see who reacted. Also, the creation date could be made human readable like the time to expiration in the `showIfActive` function.

User

```
_id: ObjectId('674c2c2d9e0f87ce0fb9bd84')
username : "stefan"
email : "stefan@mail.com"
password : "$2a$05$C6kv1eU2wSYfSbXxHerX7.Rs8EqI.a6MFf863vWYbi3y/zmXPCQdC"
date : 2024-12-01T09:28:13.601+00:00
__v : 0
```

The username is not unique, so there could be more than one user with the same name, but for linking, the user ID is stored. Like this, buttons like `reply in direct message` or so, will still work.

Reactions and comments

The reactions and comments on posts can only be done from the post detail endpoint. The frontend can call that from the list view and only update the affected post though.

Filter and Sort

With this solution, the database stores the arrays and if the user wants to sort by interest, the app has to count and sort.

Similarily the status 'active' is calculated by the app with the help of database values and the local time of the client.

Because both of the above modify the posts that are displayed and can be filtered or sorted, the logic gets a bit messy. One sort or filter is done locally and based on that, the rest is sent to the database to handle.

> **IDEA:** The limit of posts fetched is set to 15, which is a good number of posts to show per page. The frontend pagination logic will trigger fetcing the next 15 if the user paginates.

API reference

`/`:
The welcome page. Responds with 'please log in' or if logged in, 'hello, [username]'.

Endpoints that can be accessed if authorised:

GET `post/`:
List of posts. Responds with an array of Post objects. Can be used with `filters` or `sort`, that are passed in the request body as follows: `{filters:[options]}`
options include:
`active`: type: Boolean `false|true`
`topic`: type: Schema.Types.ObjectId `"topicId"`
`authorId`: type: Schema.Types.ObjectId `"topicId"`

POST `post/new`:
Create a new post. Responds with the newly created post object. The new post is passed in the request body in the following format:

```
title: { type: String, required: true, min: 3, max: 256 },
text: { type: String, required: true, min: 3, max: 2048 },
topicId: [{ type: Schema.Types.ObjectId, required: true, ref: "topics" }],
expirationTime: { type: Number, required: true, min: 5 },
```

example:

```
{"title":"Question",  "text":"What is the coolest animal?",
"expirationTime":"8",  "topicId": "674c7965f3a45b229d3b02d9"}
```

GET `post/:id`:
Detail view of a post.

POST `post/:id/comment`:
Accepts new comments for posts if they are still active. The comments are passed in the request body in the following format:

```
{ text: { type: String, required: true, min: 3, max: 1024 }}
```

POST `post/:id/react`:
Accepts new reactions for posts if they are still active. Reactions are passed in the request body in the following format:

```
{ reaction: [options]}
```

The options are one of the two following strings: `"likes"` or `"dislikes"`

GET `topic/`:
List of all the topics. Still needs to be implemented. Curretnly responds with the string “Topic list endpoint”

GET `topic/:id`:
Detail view of a topic. Responds with a list of post that belong to the topic. Accepts the same request body options as `post/`.

POST `user/login`:
Login page. Responds with a json web token. This is implented in that way that it is easy to test/review. Accepts the following request body:

```
{email: { type: String, required: true, min: 3, max: 256 },
password: { type: String, required: true, min: 3, max: 1024 },}
```

example:

```
{"email":"user@mail.com",
"password":"TQVm4cBK"}
```

POST `user/registration`: Registration page. Responds with the new created user object. Accepts the following request body:

```
username: { type: String, required: true, min: 3, max: 256 },
email: { type: String, required: true, min: 3, max: 256 },
password: { type: String, required: true, min: 3, max: 1024 }
```

example:

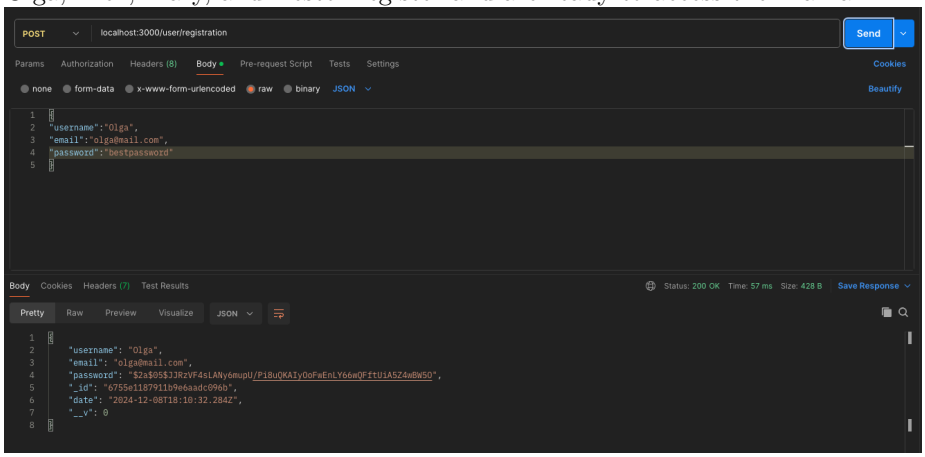
```
{"username":"user",
"email":"user@mail.com",
"password":"TQVm4cBK"}
```

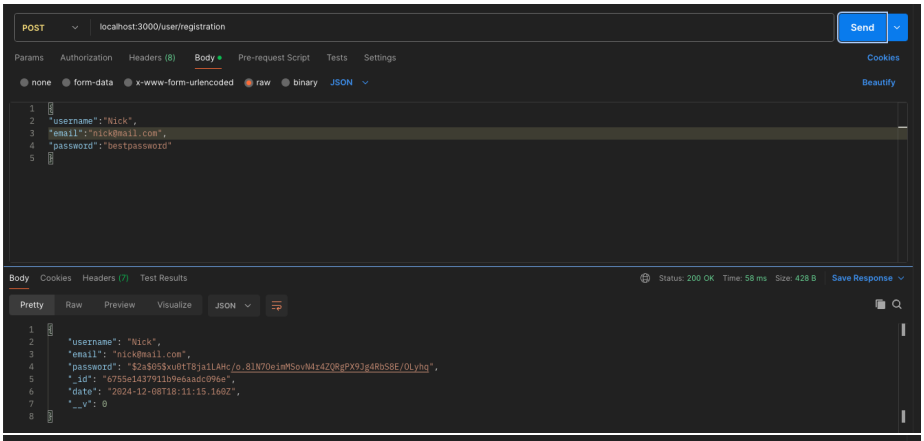
GET `user/:id`
User detail page. Still needs to be implemented. Curretnly responds with the string “User profile endpoint”.

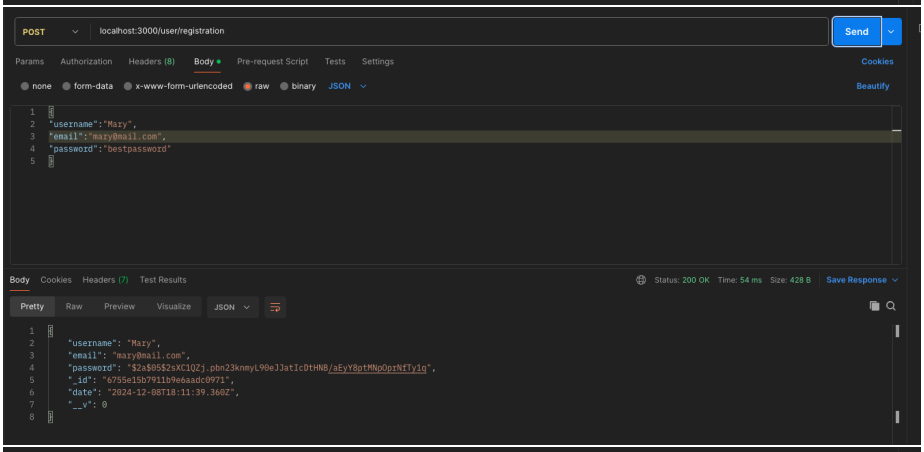
Phase D

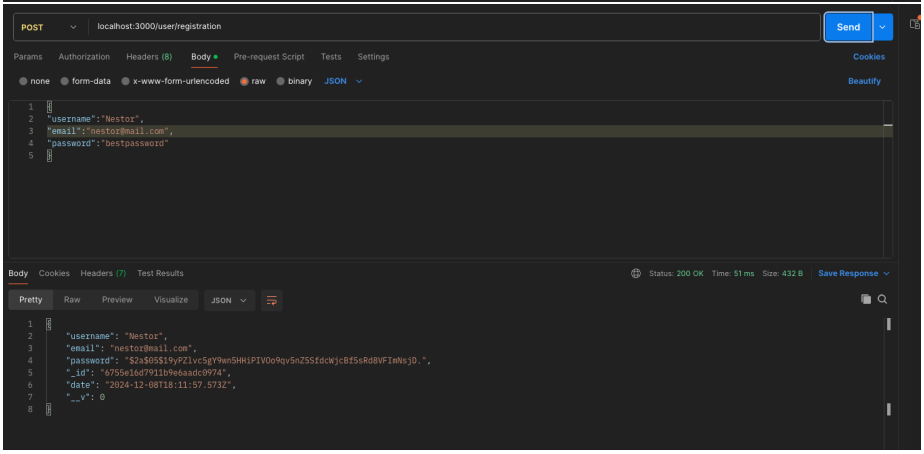
To test the application, the app was deployed in the google cloud virtual machine and then tested manually.

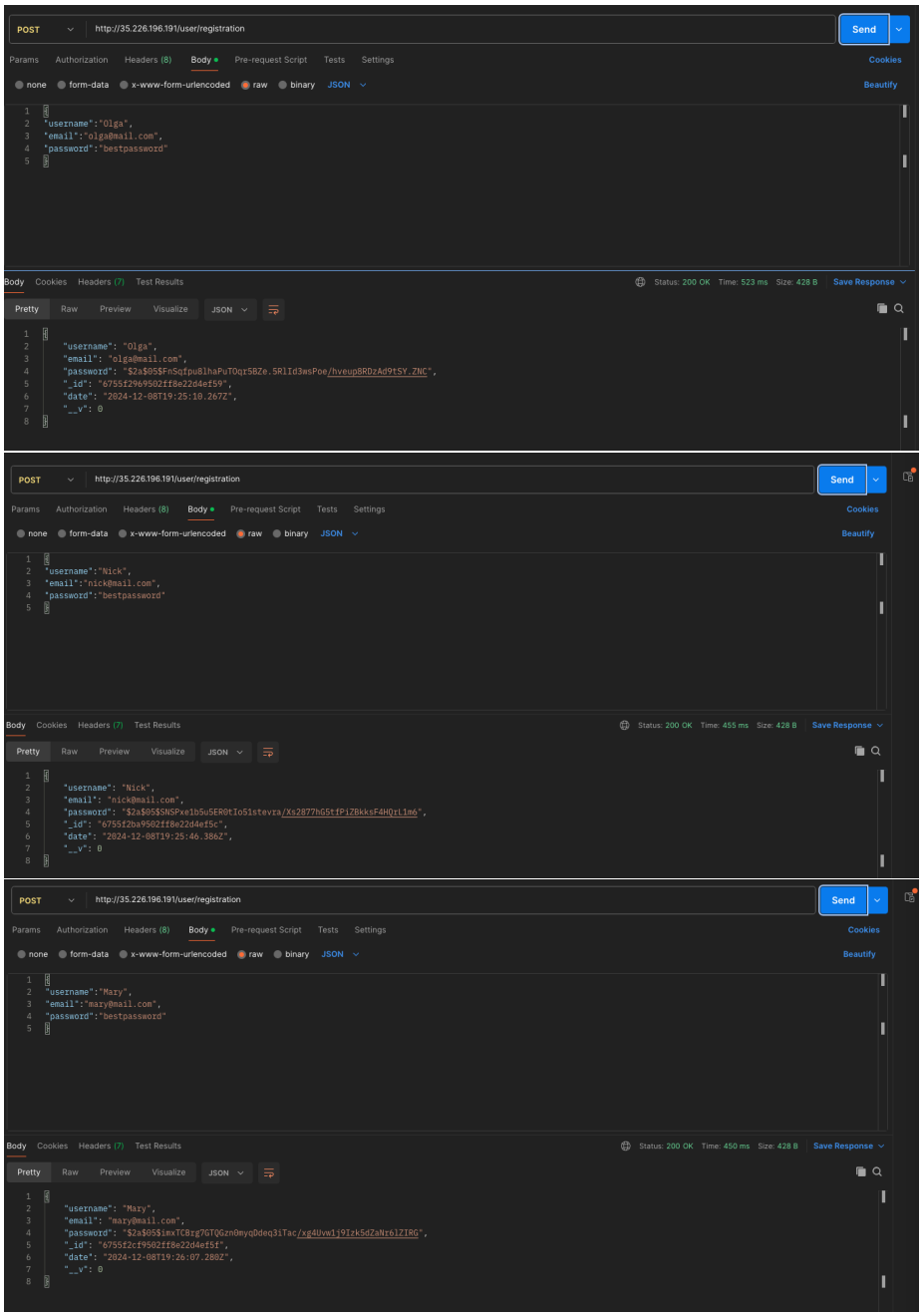
TC1:
Olga, Nick, Mary, and Nestor register and are ready to access the Piazza API.

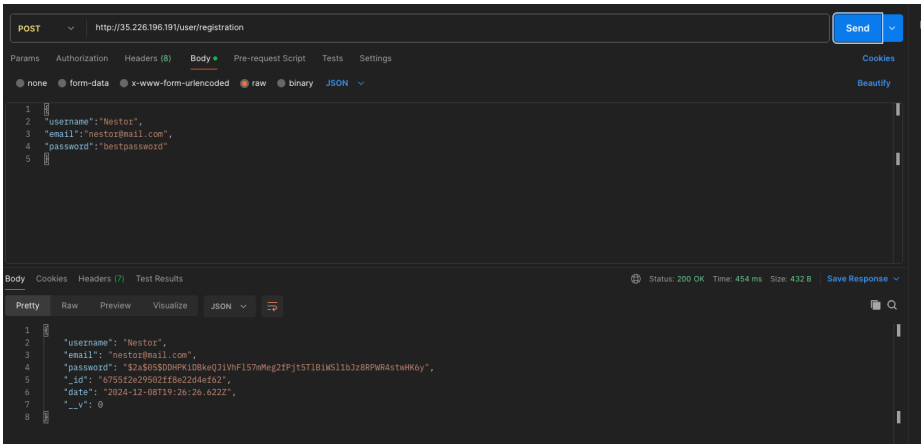



POST
localhost:3000/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Nick",
"email":"nick@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 58 ms Size: 428 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Nick",
"email": "nick@mail.com",
"password": "$2a$05$xu0tTBja1LAHc/o.8lN7OeimMSovN4r4ZQRgPX9Jg4RbS8E/OLyhq",
"_id": "6755e1437911b9e6aadc096e",
"date": "2024-12-08T18:11:15.160Z",
"__v": 0
}


POST
localhost:3000/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Mary",
"email":"mary@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 54 ms Size: 428 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Mary",
"email": "mary@mail.com",
"password": "$2a$05$2sXC1QZj.pbn23knmyL90eJJatIcDtHN8/aEyY8ptMNpOprNfTy1q",
"_id": "6755e15b7911b9e6aadc0971",
"date": "2024-12-08T18:11:39.360Z",
"__v": 0
}

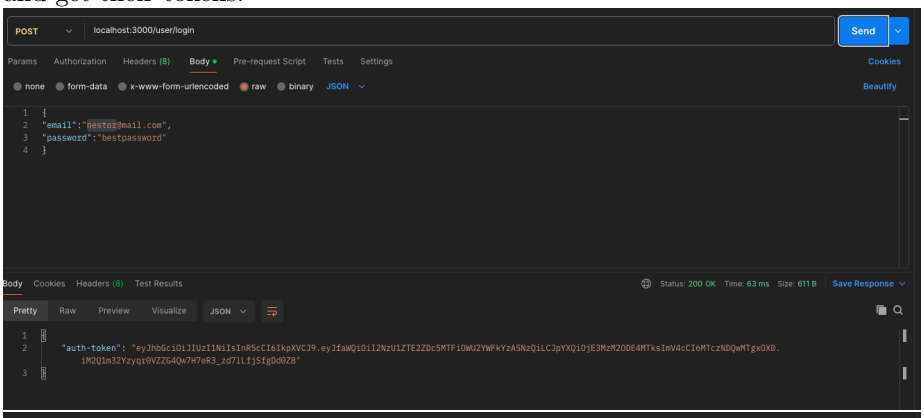

POST
localhost:3000/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Nestor",
"email":"nestor@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 51 ms Size: 432 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Nestor",
"email": "nestor@mail.com",
"password": "$2a$05$19yPZlvc5gY9wn5HHiPIVOo9qv5nZ5SfdcWjcBf5sRd8VFImNsjD.",
"_id": "6755e16d7911b9e6aadc0974",
"date": "2024-12-08T18:11:57.573Z",
"__v": 0
}

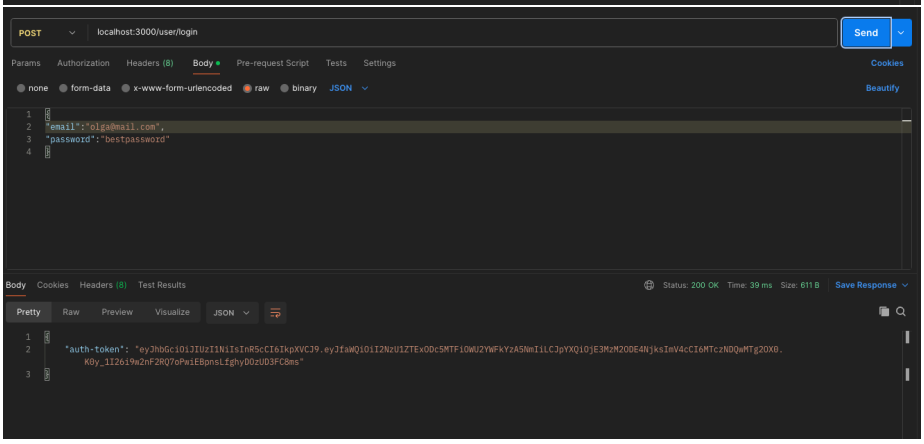

POST
http://35.226.196.191/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Olga",
"email":"olga@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 523 ms Size: 428 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Olga",
"email": "olga@mail.com",
"password": "$2a$05$FnSqfpuBlhaPuTOqr5BZe.5RlId3wsPoe/hveup8RDzAd9tSY.ZNC",
"_id": "6755f296950ff8e22d4ef59",
"date": "2024-12-08T19:25:10.267Z",
"__v": 0
}
POST
http://35.226.196.191/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Nick",
"email":"nick@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 455 ms Size: 428 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Nick",
"email": "nick@mail.com",
"password": "$2a$05$SNSPxe1b5u5ER0tIo51stevra/Xs2877hG5tfPiZBkksF4HQrL1m6",
"_id": "6755f2ba9502ff8e22d4ef5c",
"date": "2024-12-08T19:25:46.386Z",
"__v": 0
}
POST
http://35.226.196.191/user/registration
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"username":"Mary",
"email":"mary@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 450 ms Size: 428 B
Save Response
Pretty Raw Preview Visualize JSON
{
"username": "Mary",
"email": "mary@mail.com",
"password": "$2a$05$imxTCBzg7GTQGzn0myqDdeq3iTac/xg4Uvw1j9Izk5dZaNz6lZIRG",
"_id": "6755f2cf9502ff8e22d4ef5f",
"date": "2024-12-08T19:26:07.280Z",
"__v": 0
}

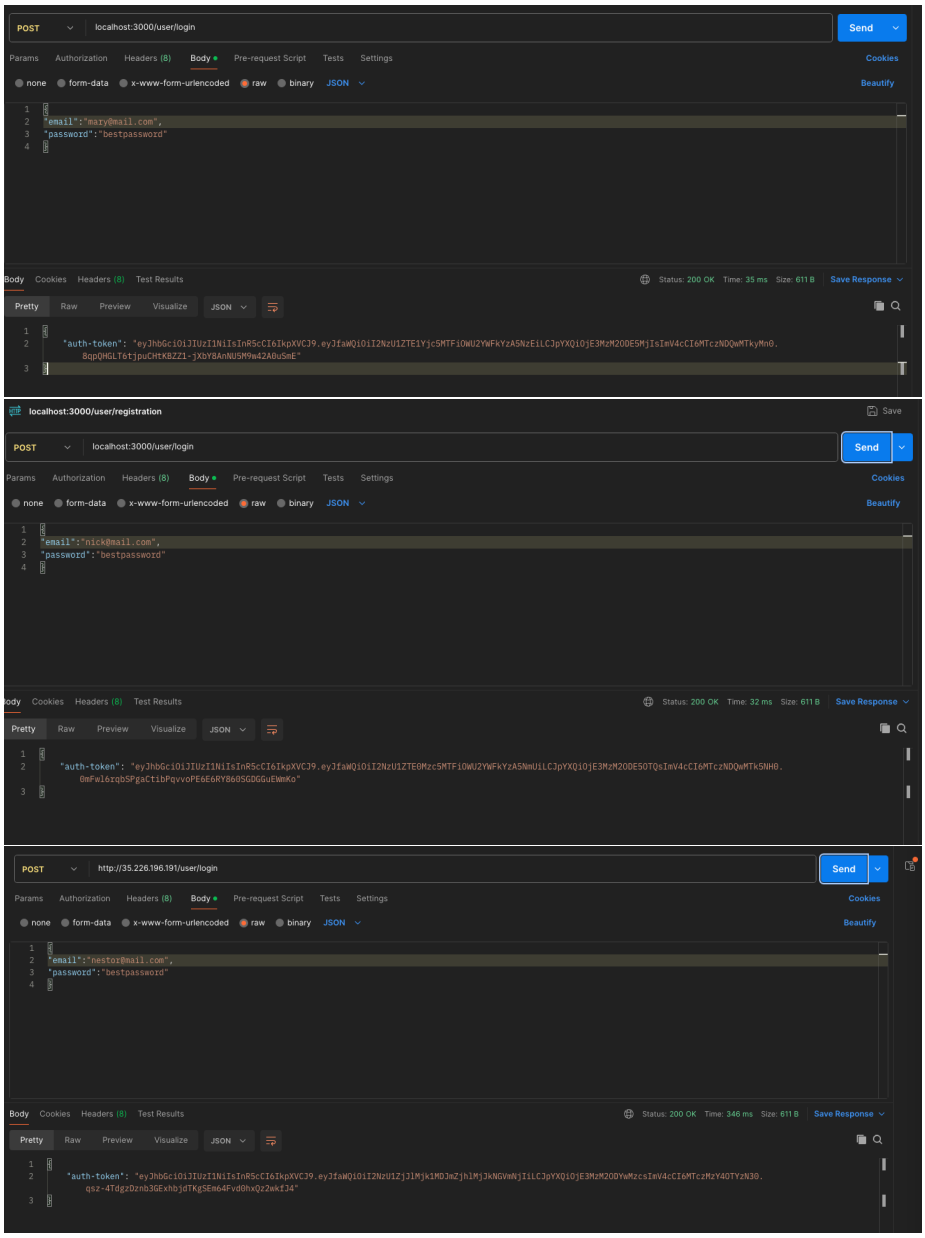


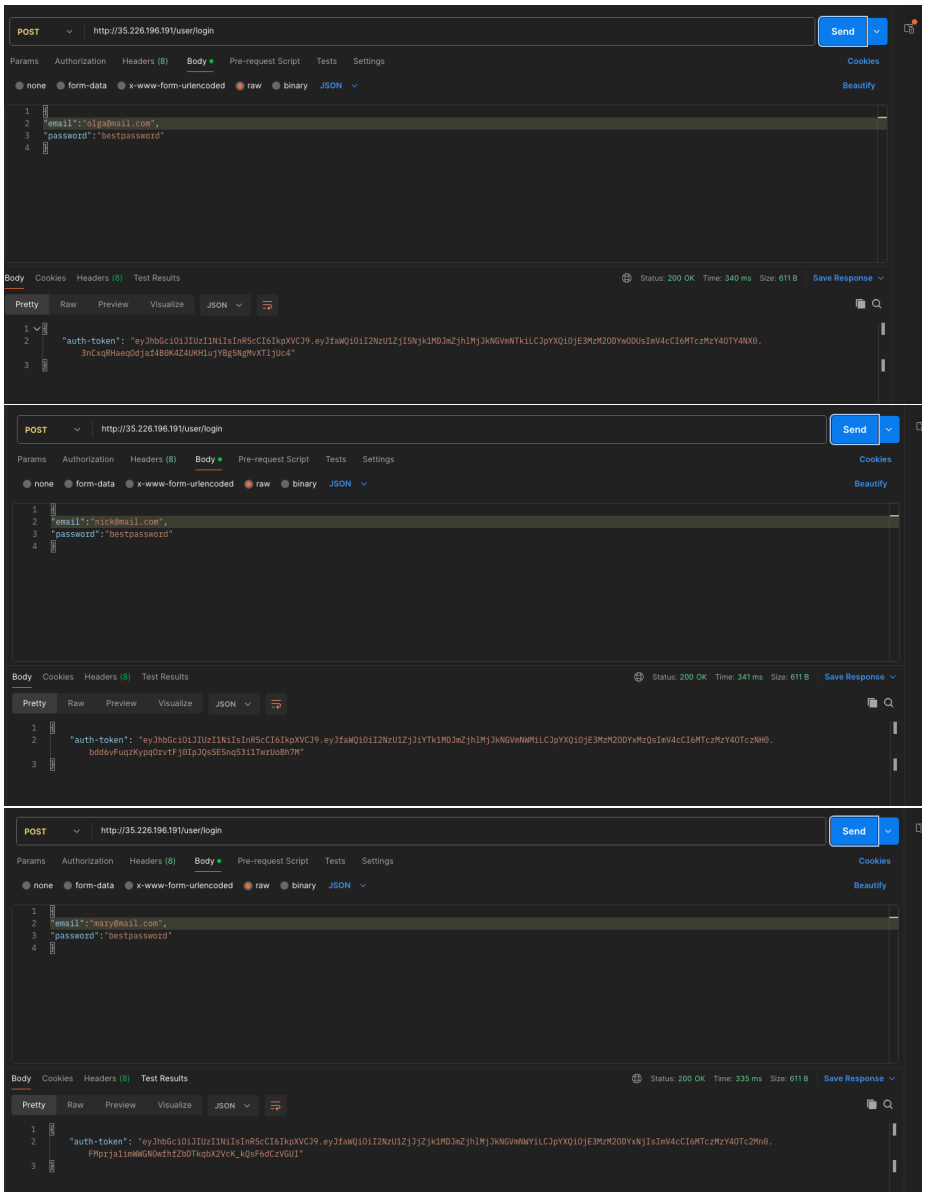
TC2:

Olga, Nick, Mary, and Nestor use the oAuth v2 authorisation service to register and get their tokens.

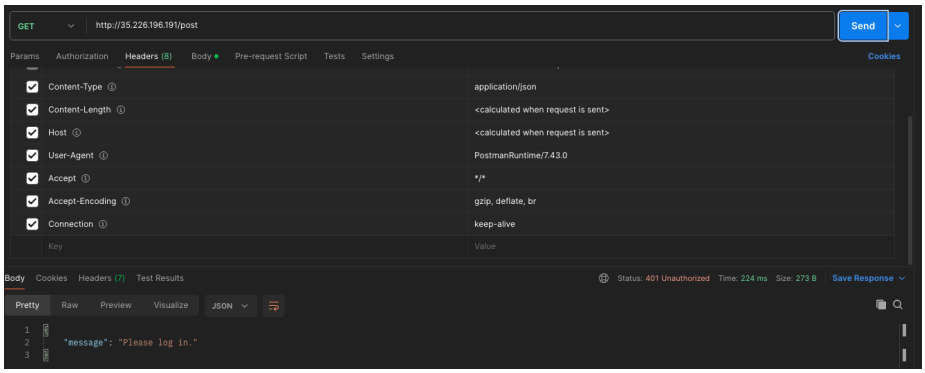




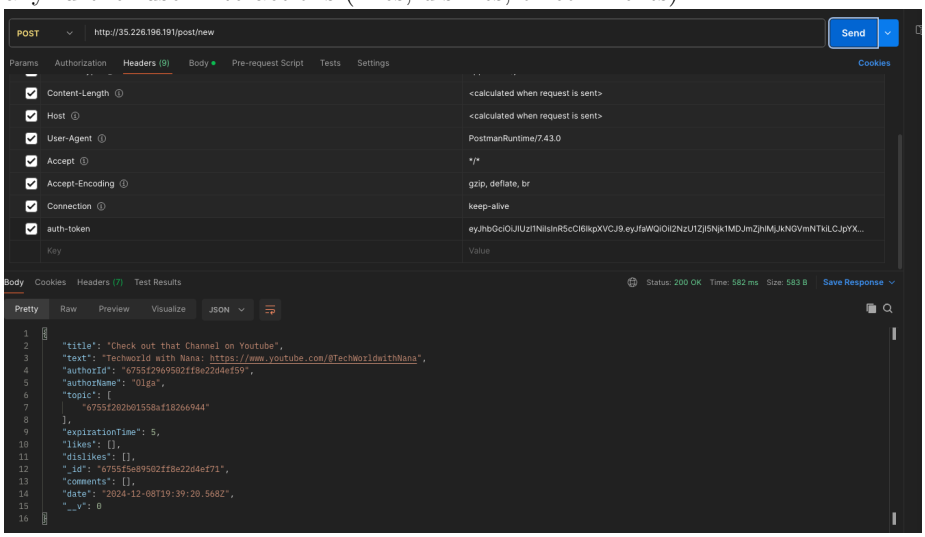

POST
localhost:3000/user/login
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"email":"mary@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (8) Test Results
Status: 200 OK Time: 35 ms Size: 611 B
Save Response
Pretty Raw Preview Visualize JSON
{
"auth-token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfaWQiOiI2NzU1ZTE1Yjc5MTFiOWU2YWFkYzA5NzEiLCJpYXQiOjE3MzM2ODE5MjIsImV4cCI6MTczNDQwMTkyMn0.
8qpQHGLT6tjpuCHtKBZZ1-jXbY8AnNU5M9w4ZAUuSmE"
}
localhost:3000/user/registration
Save
POST
localhost:3000/user/login
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"email":"nick@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (8) Test Results
Status: 200 OK Time: 32 ms Size: 611 B
Save Response
Pretty Raw Preview Visualize JSON
{
"auth-token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfaWQiOiI2NzU1ZTE0Mzc5MTFiOWU2YWFkYzA5NmUiLCJpYXQiOjE3MzM2ODE5OTQsImV4cCI6MTczNDQwMTk5NH0.
0mFwl6zqbSPgaCtibPqvvoPE6E6RY8b0SGDGGuEWmKo"
}
POST
http://35.226.196.191/user/login
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"email":"nestor@mail.com",
"password":"bestpassword"
}
Body Cookies Headers (8) Test Results
Status: 200 OK Time: 346 ms Size: 611 B
Save Response
Pretty Raw Preview Visualize JSON
{
"auth-token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfaWQiOiI2NzU1ZjJlMjk1MDJmZjhlMjJkNGVmNjIiLCJpYXQiOjE3MzM2ODYwMzcsImV4cCI6MTczMzY4OTYzN30.
qsz-4TdgzDznb3GExhbjdTKgSEm64Fvd0hxQz2wkfJ4"
}



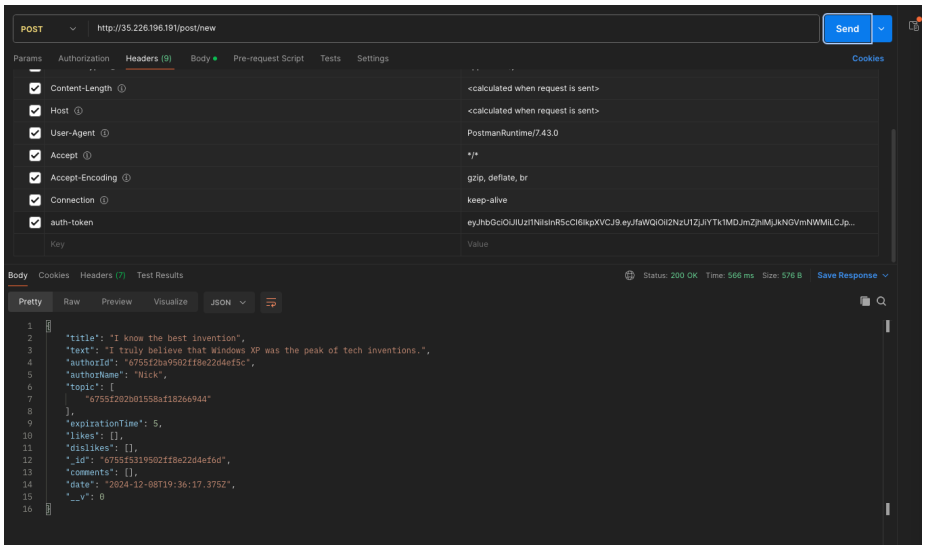
TC3:
Olga makes a call to the API without using her token. This call should be unsuccessful as the user is unauthorised.



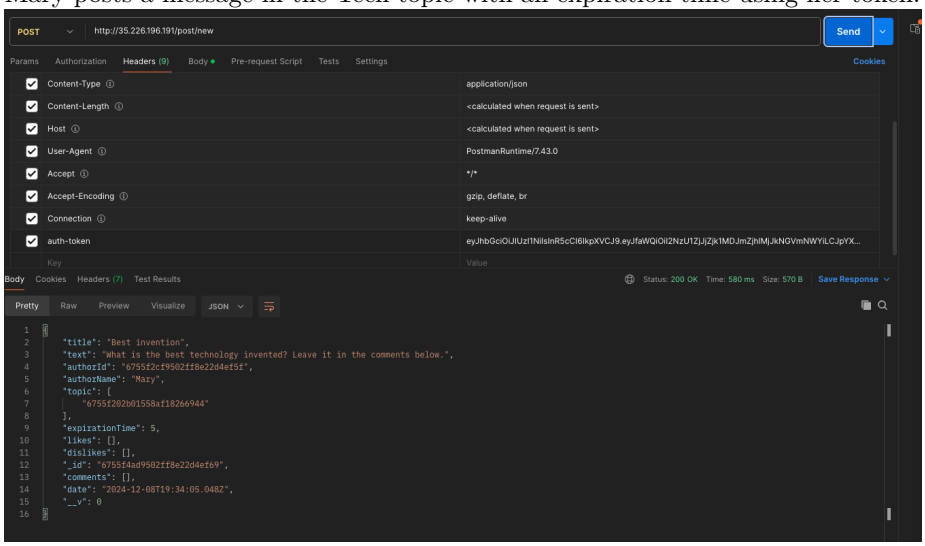
TC4:
Olga posts a message in the Tech topic with an expiration time (e.g. 5 minutes) using her token. After the end of the expiration time, the message will not accept any further user interactions (likes, dislikes, or comments).



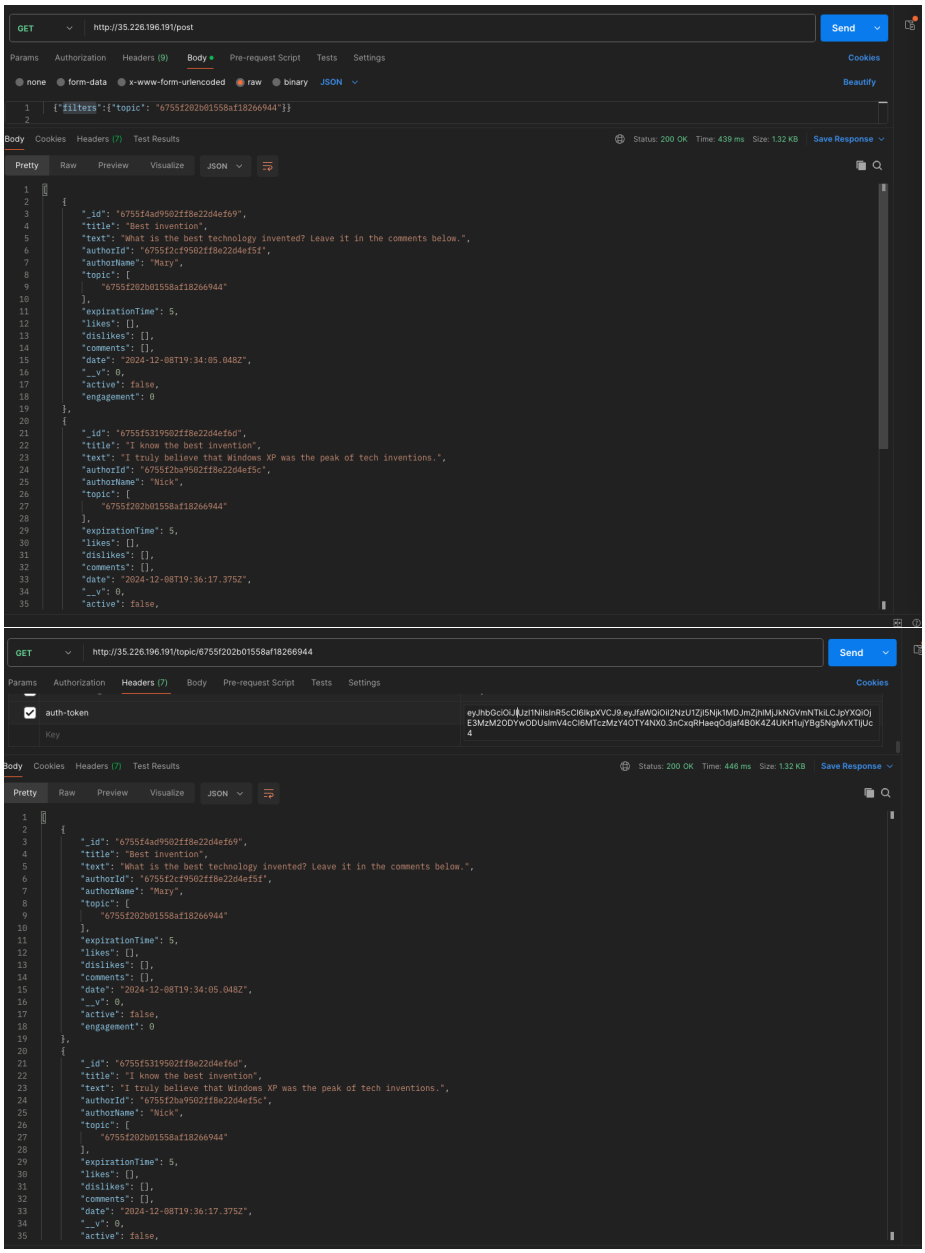
TC5:
Nick posts a message in the Tech topic with an expiration time using his token.



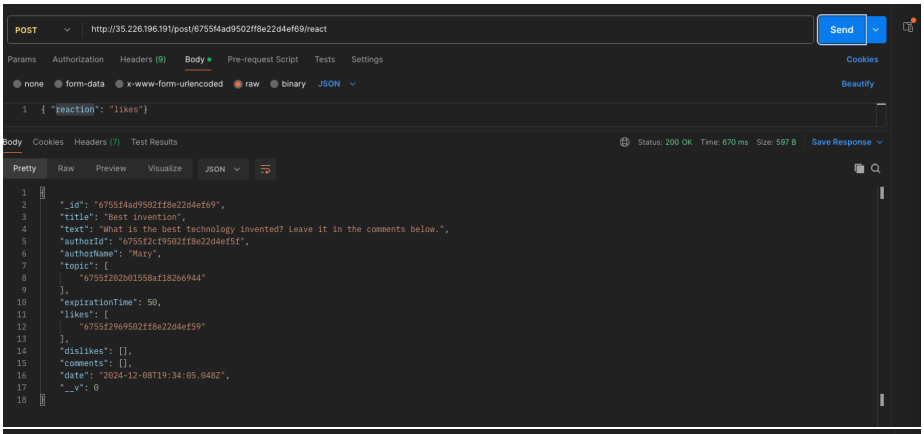
TC6:
Mary posts a message in the Tech topic with an expiration time using her token.

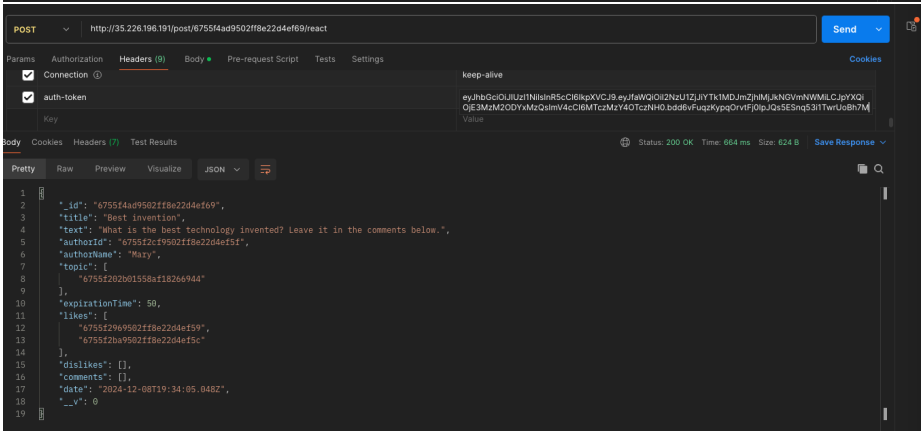


TC7:
Nick and Olga browse all the available posts in the Tech topic; three posts should have zero likes, zero dislikes, and no comments.

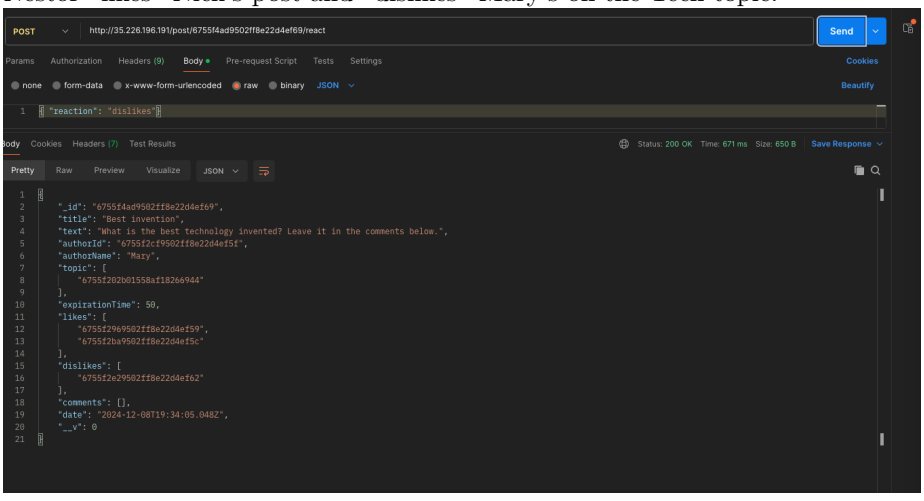


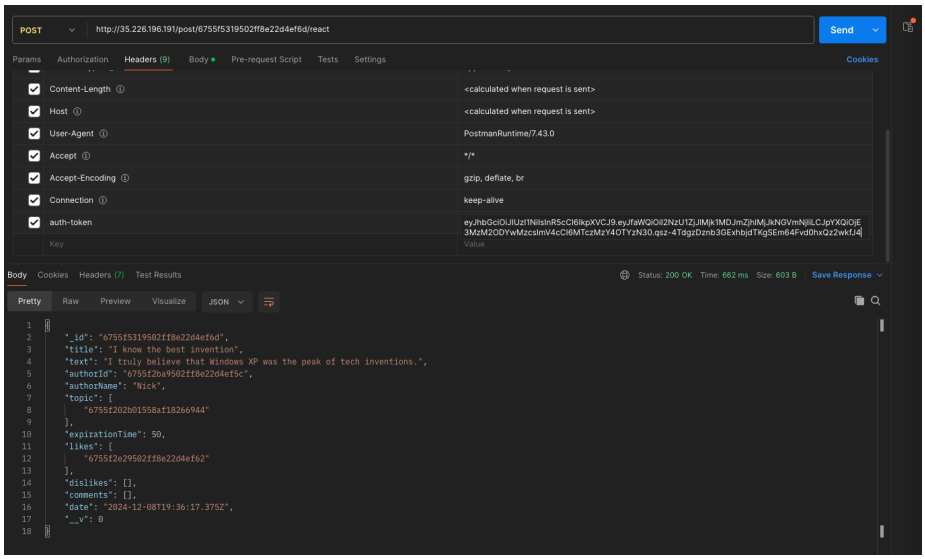
TC8:
Nick and Olga “like” Mary’s post on the Tech topic.





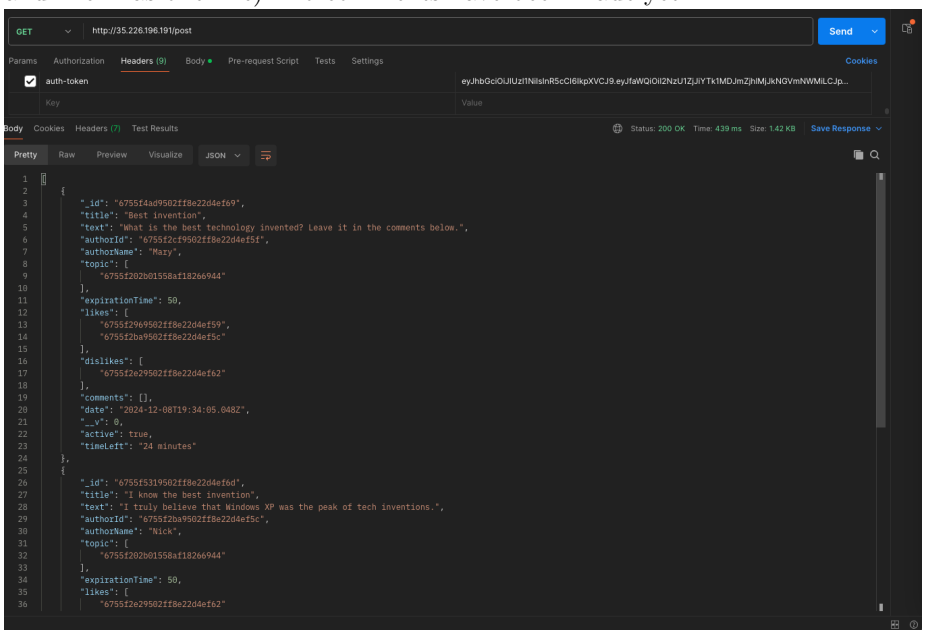
TC9:
Nestor “likes” Nick’s post and “dislikes” Mary’s on the Tech topic.





TC10:

Nick browses all the available posts on the Tech topic; at this stage, he can see the number of likes and dislikes for each post (Mary has two likes and one dislike, and Nick has one like). No comments have been made yet.

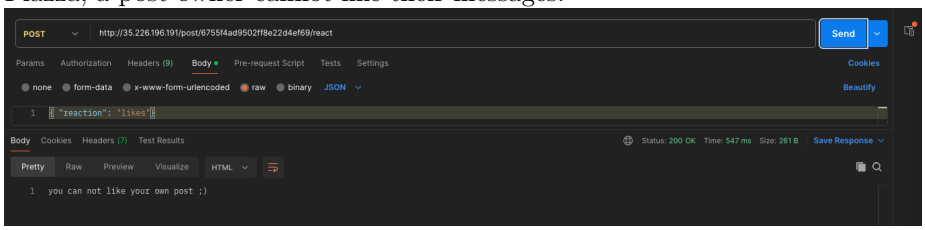



```
[
    {
        "_id": "6755f4ad9502ff8e22d4ef69",
        "title": "Best invention",
```

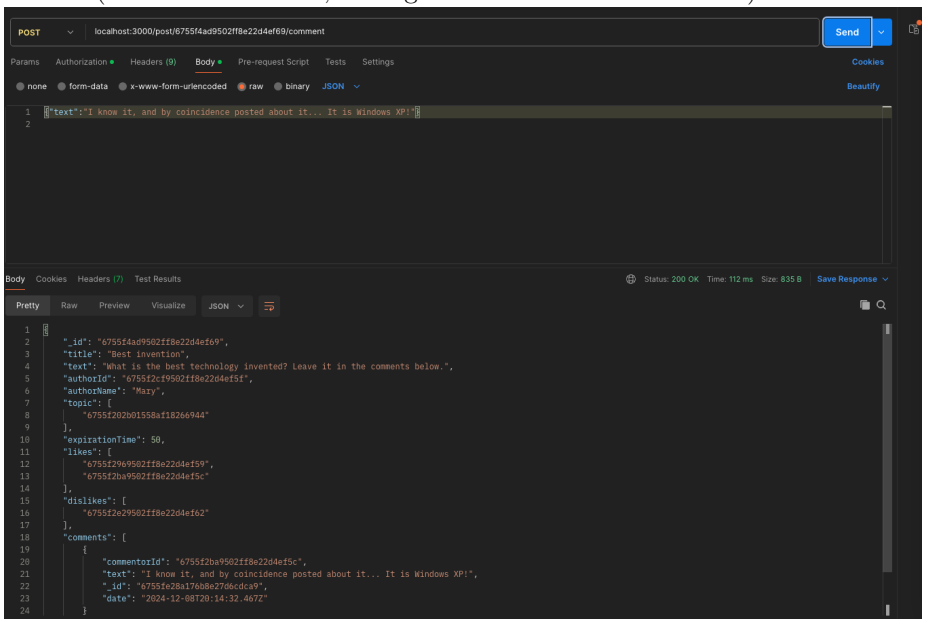
```
    "text": "What is the best technology invented? Leave it in the comments below.",
    "authorId": "6755f2cf9502ff8e22d4ef5f",
    "authorName": "Mary",
    "topic": [
        "6755f202b01558af18266944"
    ],
    "expirationTime": 50,
    "likes": [
        "6755f2969502ff8e22d4ef59",
        "6755f2ba9502ff8e22d4ef5c"
    ],
    "dislikes": [
        "6755f2e29502ff8e22d4ef62"
    ],
    "comments": [],
    "date": "2024-12-08T19:34:05.048Z",
    "__v": 0,
    "active": true,
    "timeLeft": "24 minutes"
},
{
    "_id": "6755f5319502ff8e22d4ef6d",
    "title": "I know the best invention",
    "text": "I truly believe that Windows XP was the peak of tech inventions.",
    "authorId": "6755f2ba9502ff8e22d4ef5c",
    "authorName": "Nick",
    "topic": [
        "6755f202b01558af18266944"
    ],
    "expirationTime": 50,
    "likes": [
        "6755f2e29502ff8e22d4ef62"
    ],
    "dislikes": [],
    "comments": [],
    "date": "2024-12-08T19:36:17.375Z",
    "__v": 0,
    "active": true,
    "timeLeft": "26 minutes"
},
{
    "_id": "6755f5e89502ff8e22d4ef71",
    "title": "Check out that Channel on Youtube",
    "text": "Techworld with Nana: https://www.youtube.com/@TechWorldwithNana",
    "authorId": "6755f2969502ff8e22d4ef59",
    "authorName": "Olga",
```

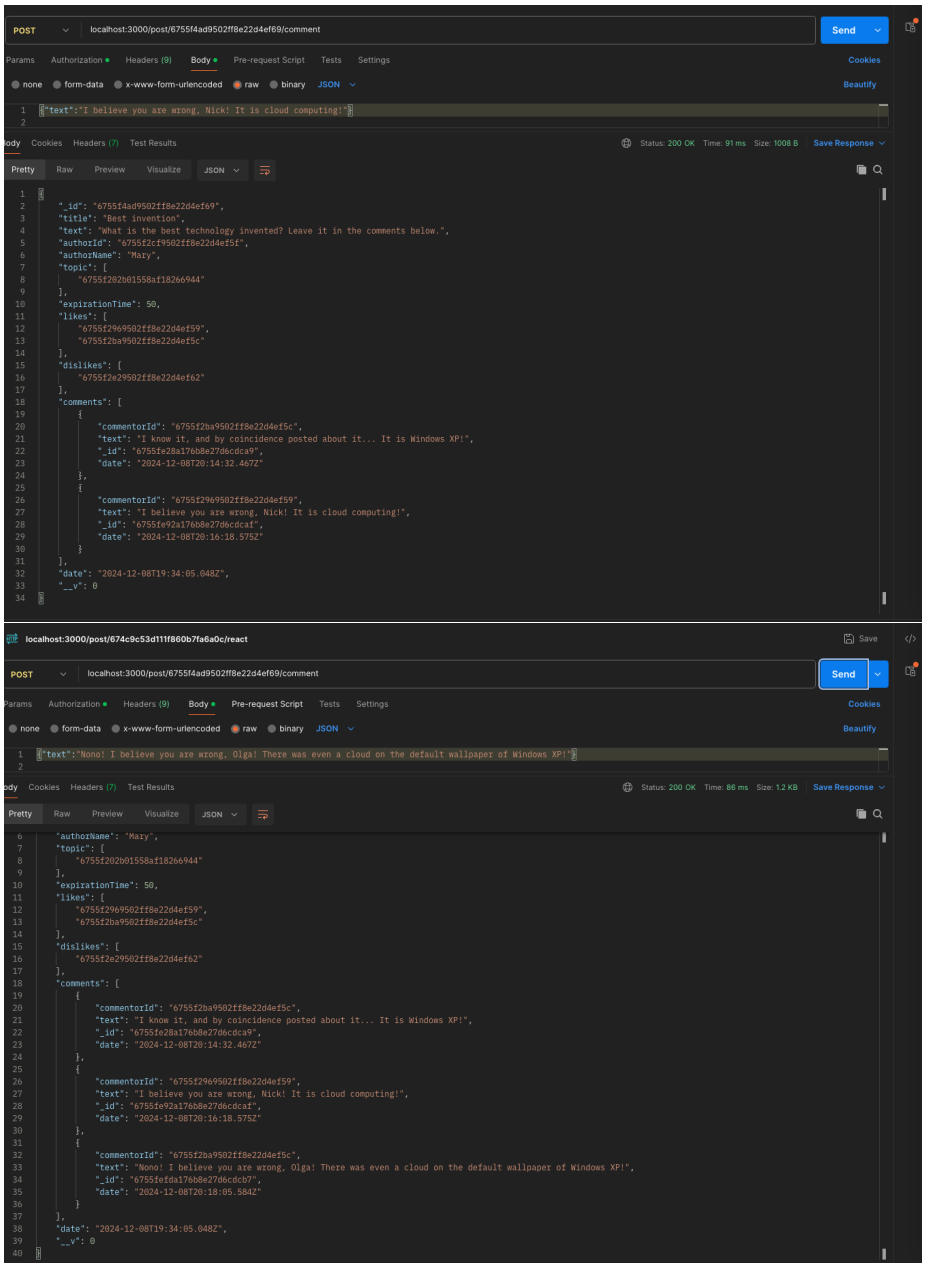
```
        "topic": [
            "6755f202b01558af18266944"
        ],
        "expirationTime": 5,
        "likes": [],
        "dislikes": [],
        "comments": [],
        "date": "2024-12-08T19:39:20.568Z",
        "__v": 0,
        "active": false
    }
]
```

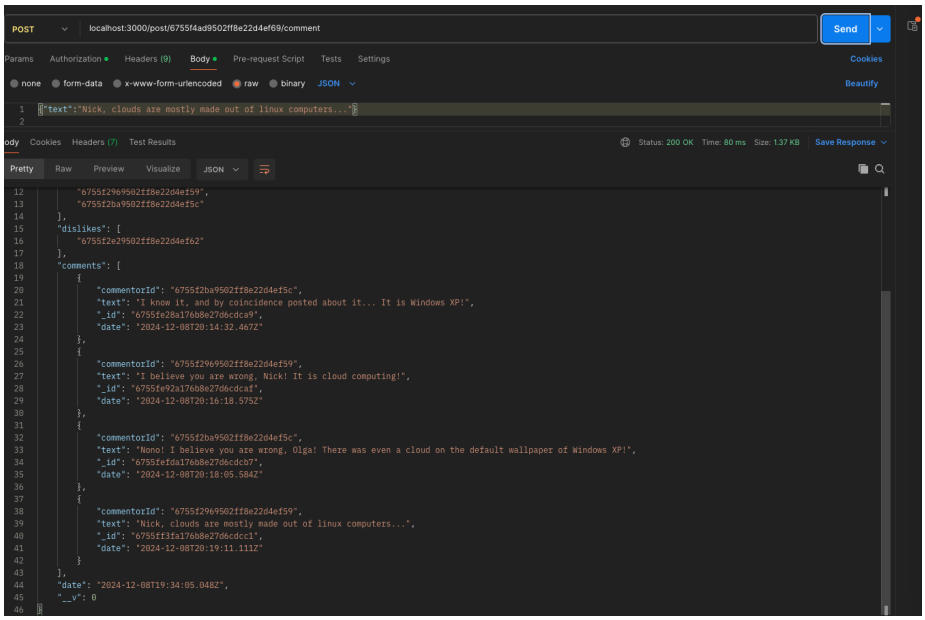

TC11:
Mary likes her post on the Tech topic. This call should be unsuccessful; in Piazza, a post owner cannot like their messages.



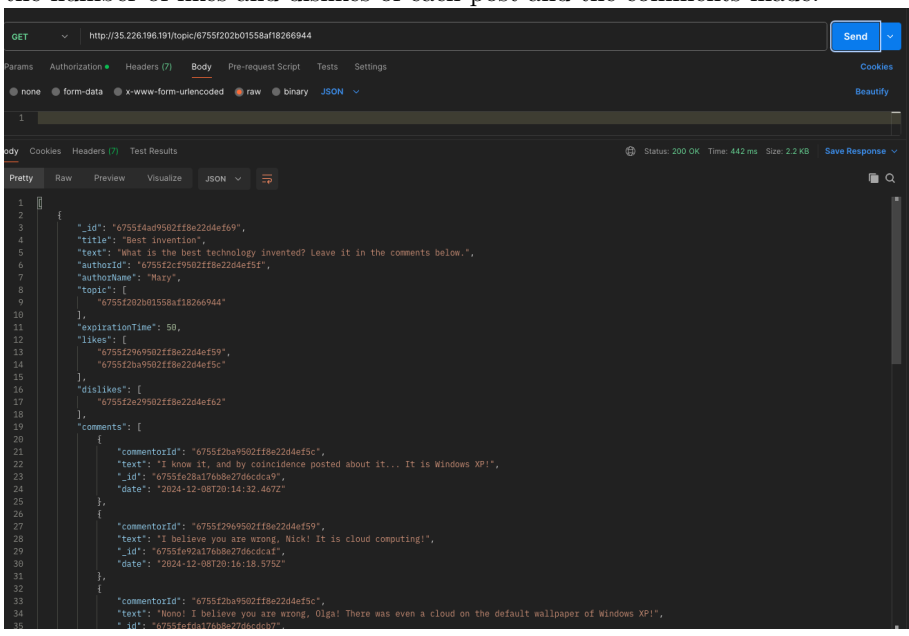
TC12:
Nick and Olga comment on Mary's post on the Tech topic in a round-robin fashion (one after the other, adding at least two comments each).




POST
localhost:3000/post/6755f4ad9502ff8e22d4ef69/comment
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{"text":"I believe you are wrong, Nick! It is cloud computing!"}
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 91 ms
Size: 1008 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"_id": "6755f4ad9502ff8e22d4ef69",
"title": "Best invention",
"text": "What is the best technology invented? Leave it in the comments below.",
"authorId": "6755f2cf9502ff8e22d4ef5f",
"authorName": "Mary",
"topic": [
"6755f202b01558af18266944"
],
"expirationTime": 50,
"likes": [
"6755f2969502ff8e22d4ef59",
"6755f2ba9502ff8e22d4ef5c"
],
"dislikes": [
"6755f2e29502ff8e22d4ef62"
],
"comments": [
{
"commentorId": "6755f2ba9502ff8e22d4ef5c",
"text": "I know it, and by coincidence posted about it... It is Windows XP!",
"_id": "6755fe28a176b8e27d6cdca9",
"date": "2024-12-08T20:14:32.467Z"
},
{
"commentorId": "6755f2969502ff8e22d4ef59",
"text": "I believe you are wrong, Nick! It is cloud computing!",
"_id": "6755fe92a176b8e27d6cdcaf",
"date": "2024-12-08T20:16:18.575Z"
}
],
"date": "2024-12-08T19:34:05.048Z",
"__v": 0
}
localhost:3000/post/674c9c53d111f860b7fa6a0c/react
Save
POST
localhost:3000/post/6755f4ad9502ff8e22d4ef69/comment
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{"text":"Nono! I believe you are wrong, Olga! There was even a cloud on the default wallpaper of Windows XP!"}
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 86 ms
Size: 1.2 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
'authorName': 'Mary',
"topic": [
"6755f202b01558af18266944"
],
"expirationTime": 50,
"likes": [
"6755f2969502ff8e22d4ef59",
"6755f2ba9502ff8e22d4ef5c"
],
"dislikes": [
"6755f2e29502ff8e22d4ef62"
],
"comments": [
{
"commentorId": "6755f2ba9502ff8e22d4ef5c",
"text": "I know it, and by coincidence posted about it... It is Windows XP!",
"_id": "6755fe28a176b8e27d6cdca9",
"date": "2024-12-08T20:14:32.467Z"
},
{
"commentorId": "6755f2969502ff8e22d4ef59",
"text": "I believe you are wrong, Nick! It is cloud computing!",
"_id": "6755fe92a176b8e27d6cdcaf",
"date": "2024-12-08T20:16:18.575Z"
},
{
"commentorId": "6755f2ba9502ff8e22d4ef5c",
"text": "Nono! I believe you are wrong, Olga! There was even a cloud on the default wallpaper of Windows XP!",
"_id": "6755fefda176b8e27d6cdcb7",
"date": "2024-12-08T20:18:05.584Z"
}
],
"date": "2024-12-08T19:34:05.048Z",
"__v": 0
}



TC13:
Nick browses all the available posts in the Tech topic; at this stage, he can see the number of likes and dislikes of each post and the comments made.

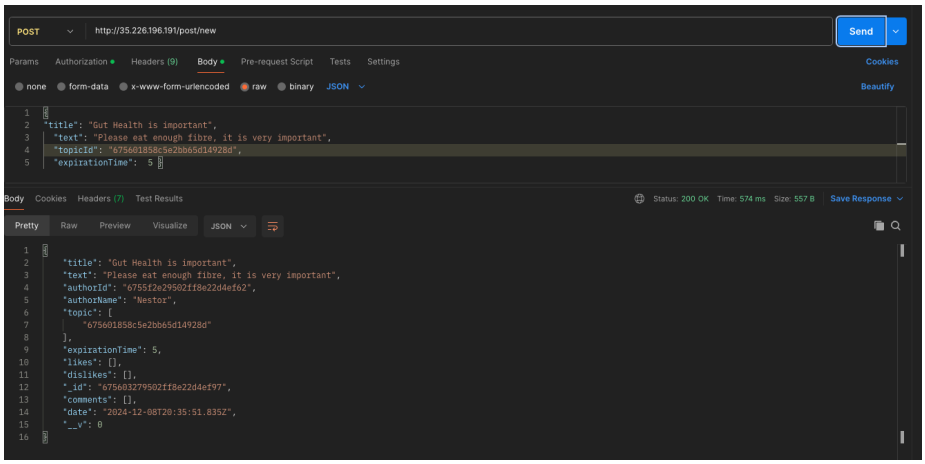



```
[
    {
        "_id": "6755f4ad9502ff8e22d4ef69",
```

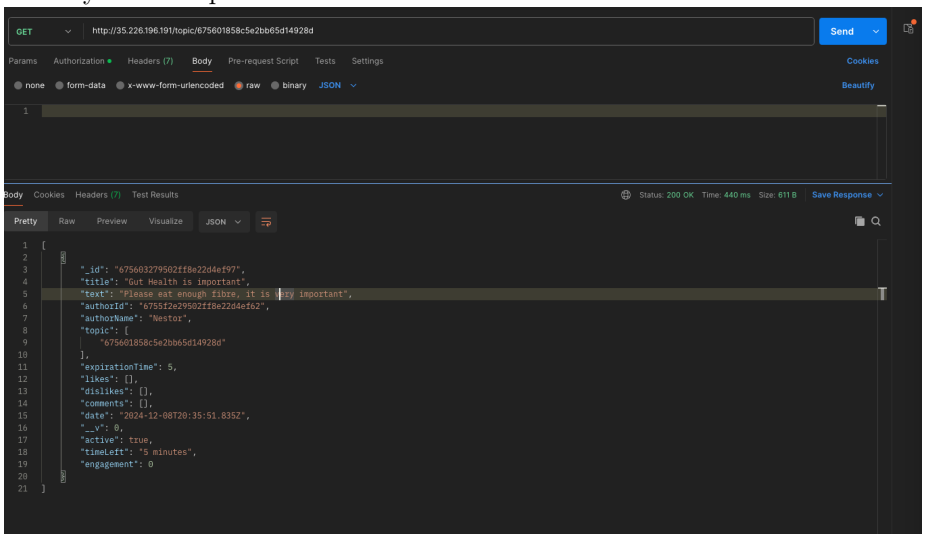
```
"title": "Best invention",
"text": "What is the best technology invented? Leave it in the comments below.",
"authorId": "6755f2cf9502ff8e22d4ef5f",
"authorName": "Mary",
"topic": [
    "6755f202b01558af18266944"
],
"expirationTime": 50,
"likes": [
    "6755f2969502ff8e22d4ef59",
    "6755f2ba9502ff8e22d4ef5c"
],
"dislikes": [
    "6755f2e29502ff8e22d4ef62"
],
"comments": [
    {
        "commentorId": "6755f2ba9502ff8e22d4ef5c",
        "text": "I know it, and by coincidence posted about it... It is Windows XP!'
        "_id": "6755fe28a176b8e27d6cdca9",
        "date": "2024-12-08T20:14:32.467Z"
    },
    {
        "commentorId": "6755f2969502ff8e22d4ef59",
        "text": "I believe you are wrong, Nick! It is cloud computing!",
        "_id": "6755fe92a176b8e27d6cdcaf",
        "date": "2024-12-08T20:16:18.575Z"
    },
    {
        "commentorId": "6755f2ba9502ff8e22d4ef5c",
        "text": "Nono! I believe you are wrong, Olga! There was even a cloud on the
        "_id": "6755fefda176b8e27d6cdcb7",
        "date": "2024-12-08T20:18:05.584Z"
    },
    {
        "commentorId": "6755f2969502ff8e22d4ef59",
        "text": "Nick, clouds are mostly made out of linux computers...",
        "_id": "6755ff3fa176b8e27d6cdcc1",
        "date": "2024-12-08T20:19:11.111Z"
    }
],
"date": "2024-12-08T19:34:05.048Z",
"__v": 0,
"active": true,
"timeLeft": "a minute",
"engagement": 3
```

```
    },
    {
        "_id": "6755f5319502ff8e22d4ef6d",
        "title": "I know the best invention",
        "text": "I truly believe that Windows XP was the peak of tech inventions.",
        "authorId": "6755f2ba9502ff8e22d4ef5c",
        "authorName": "Nick",
        "topic": [
            "6755f202b01558af18266944"
        ],
        "expirationTime": 50,
        "likes": [
            "6755f2e29502ff8e22d4ef62"
        ],
        "dislikes": [],
        "comments": [],
        "date": "2024-12-08T19:36:17.375Z",
        "__v": 0,
        "active": true,
        "timeLeft": "3 minutes",
        "engagement": 1
    },
    {
        "_id": "6755f5e89502ff8e22d4ef71",
        "title": "Check out that Channel on Youtube",
        "text": "Techworld with Nana: https://www.youtube.com/@TechWorldwithNana",
        "authorId": "6755f2969502ff8e22d4ef59",
        "authorName": "Olga",
        "topic": [
            "6755f202b01558af18266944"
        ],
        "expirationTime": 5,
        "likes": [],
        "dislikes": [],
        "comments": [],
        "date": "2024-12-08T19:39:20.568Z",
        "__v": 0,
        "active": false,
        "engagement": 0
    }
]
```

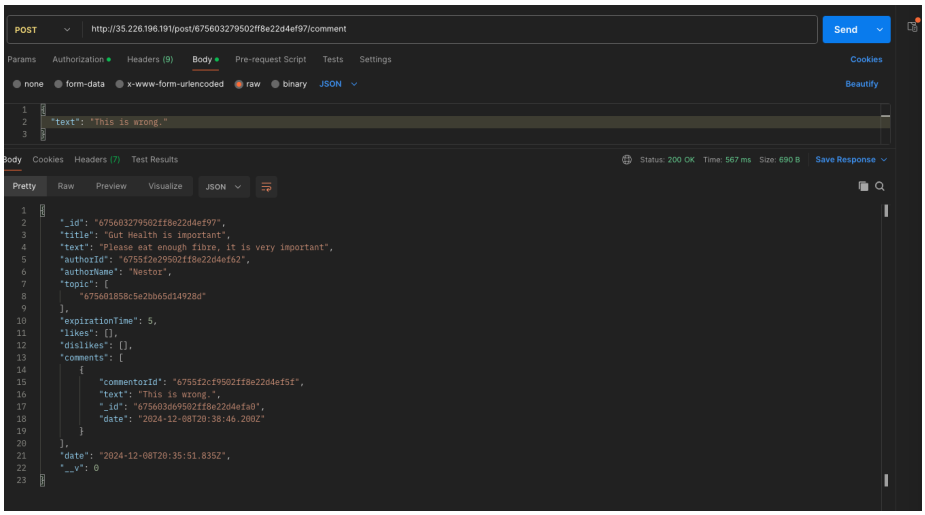

TC14:
Nestor posts a message on the Health topic with an expiration time using her token.



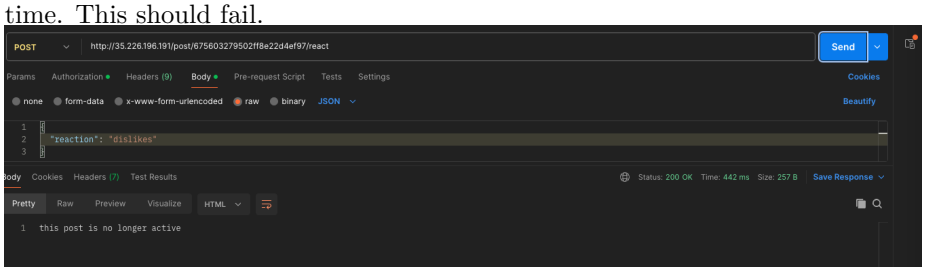
TC15:
Mary browses all the available posts on the Health topic; at this stage, she can see only Nestor's post.



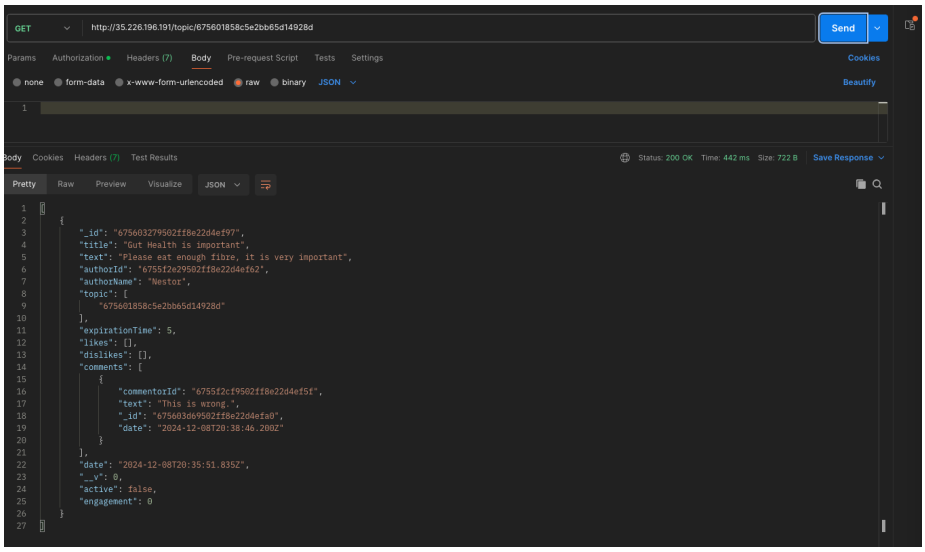
TC16:
Mary posts a comment in Nestor's message on the Health topic.



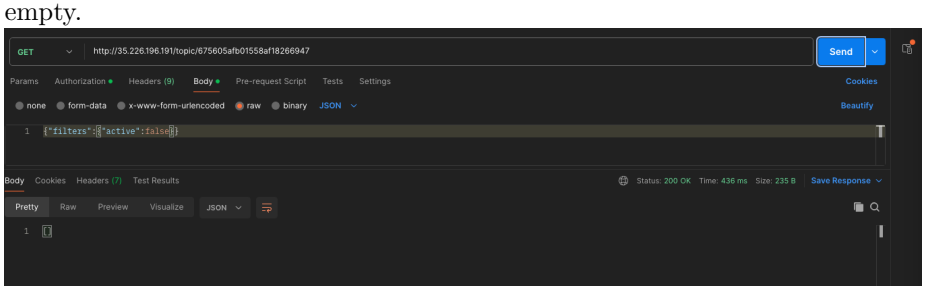
TC17:
Mary dislikes Nestor's message on the Health topic after the end of post-expiration time. This should fail.



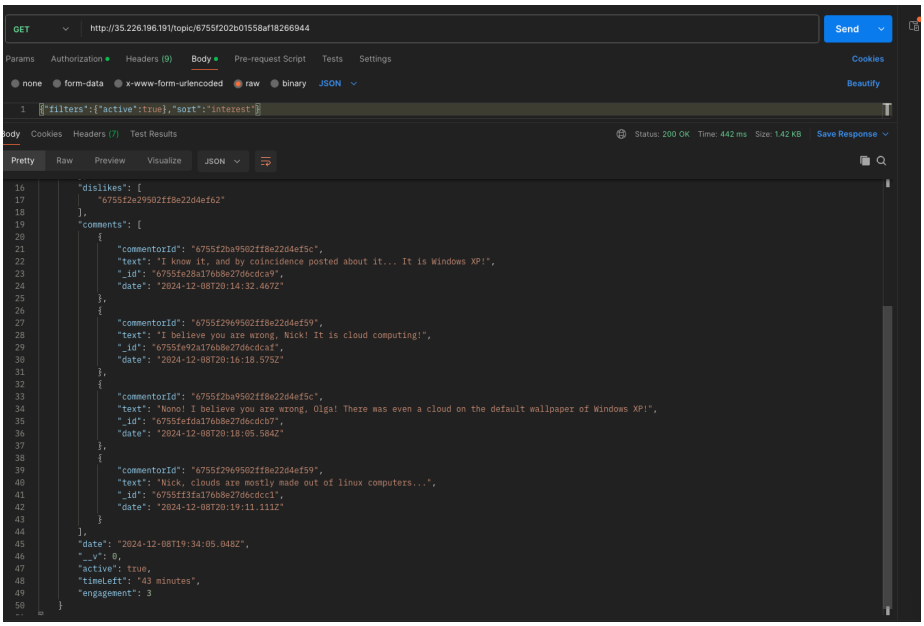
TC18:
Nestor browses all the messages on the Health topic. There should be only one post (his own) with one comment (Mary's).



TC19:
Nick browses all the expired messages on the Sports topic. These should be empty.



TC20:
Nestor queries for an active post with the highest interest (maximum number of likes and dislikes) in the Tech topic. This should be Mary's post.



Phase E

For the testing, the app was deployed into a google cloud virtual machine with almost the same method as described above in Phase A: From my local, I push code to Github, and there I manually trigger a Github Action, that lets the docker cloud build the image. After that, the image is pulled from the cloud VM, that has Docker installed and a docker user set up. The only differenct to Phase A is, that now there is an app deployed that needs to handle tokens and connect to the database. As I am not pushing the .env file to github, the secrets must be provided differently: To provide the env variables needed, I passed them to the Docker container using the –env flag: `docker run --env MONGODB_URI"<the actual string>" --env JWT_SECRET="<the actual secret>" -p 80:3000 meierstefan/piazza-api` There are better ways, but for this one time deployment it is fine like this.

Phase F

5 Replicas

In kubernetes, secrets can not simply be passed as env variables, because there are multiple containers and maybe we will add more later. To provide the secret in the simplest way, I created Kubernetes secrets as follows:

```
kubectl create secret generic database-connection \
--from-literal=MONGODB_URI='<actual string that the mongodb dashboard provides>'
```

```
kubectl create secret generic json-web-token-secret \
--from-literal=JWT_SECRET='<actual secret>'
```

Then everything was ready and the fife nodes could be started like we learned in class (cc/Class-6 at master · warestack/cc, no date, p. 6). First the deployment yaml file is created with `vim` directly in the google cloud console.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: piazza-api-deployment
  labels:
    app: piazza-api
spec:
  replicas: 5
  selector:
    matchLabels:
      app: piazza-api
  template:
    metadata:
      labels:
        app: piazza-api
    spec:
      containers:
      - name: piazza-api
        image: meierstefan/piazza-api:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 3000
        envFrom:
        - secretRef:
            name: json-web-token-secret
        - secretRef:
            name: database-connection
```

The simple steps are explained one by one in this article (Kubernetes Deployment YAML File with Examples, no date). Interesting is the `imagePullPolicy`: Here (Service, no date) the kubernetes docs explain what is going on. We have set it so `Always`, which means that everytime a new container is launched, kubernetes checks, if there is a new image in the docker registry. If there is a new one, it pulls it to the virtual machine. But if the one in the cache is the exactly same already, it does not pull the image because it can use the cache.

To start the replicas, we tell `kubectl` to apply the file.

```
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ kubectl apply -f piazza-api-deployment.yaml
```

To make sure that they are alive, we tell it to get the pods.

```
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ cat piazza-api-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: piazza-api-deployment
  labels:
    app: piazza-api
spec:
  replicas: 5
  selector:
    matchLabels:
      app: piazza-api
  template:
    metadata:
      labels:
        app: piazza-api
    spec:
      containers:
      - name: piazza-api
        image: meierstefan/piazza-api:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 3000
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
piazza-api-deployment-84c6dff8d7-2hjgf   1/1     Running   0          76s
piazza-api-deployment-84c6dff8d7-fnsjb   1/1     Running   0          76s
piazza-api-deployment-84c6dff8d7-h8jbx   1/1     Running   0          76s
piazza-api-deployment-84c6dff8d7-lgngt   1/1     Running   0          76s
piazza-api-deployment-84c6dff8d7-n48gq   1/1     Running   0          3m16s
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$
```

> **IDEA:** To make the setup easier, it would be a good idea to put the secrets into the secret-manager (Secret Manager, no date) and then they could be accessed from terraform. Then, it is no longer neccesary to manually start the pods with `kubectl`.

Loadbalancer

With the load balancer, we make the pods accessible. Like above, we need this file first:

```
apiVersion: v1
kind: Service
metadata:
  name: piazza-api-service
  labels:
    app: piazza-api-service
spec:
  type: LoadBalancer
  ports:
  - name: http
    port: 80
    protocol: TCP
    targetPort: 3000
  selector:
```

```
    app: piazza-api
  sessionAffinity: None
```

```
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ vim piazza-api-service.yaml
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ cat piazza-api-
cat: piazza-api-: No such file or directory
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ cat piazza-api-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: piazza-api-service
  labels:
    app: piazza-api-service
spec:
  type: LoadBalancer
  ports:
  - name: http
    port: 80
    protocol: TCP
    targetPort: 3000
  selector:
    app: piazza-api
  sessionAffinity: None
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ kubectl apply -f piazza-api-service.yaml
service/piazza-api-service created
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ kubectl get services
NAME                TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      34.118.224.1    <none>        443/TCP        26m
piazza-api-service  LoadBalancer   34.118.238.19   <pending>     80:31608/TCP   19s
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$ kubectl get services
NAME                TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      34.118.224.1    <none>        443/TCP        28m
piazza-api-service  LoadBalancer   34.118.238.19   34.46.72.93   80:31608/TCP   104s
smeier01@cloudshell:~ (deft-bazaar-437518-v4)$
```

Figure 1: Kubernetes start loadbalancer service.

The docs (Service, no date b) explain that kubernetes does not have a load balancer, but by setting the `type` of this service to `Loadbalancer`, we will use the google cloud default load balancer. We set `sessionAffinity` to `None` because it does not matter if the request of the client is always passed to the same pod.

A Docker tutorial (2022). Available at: https://www.youtube.com/watch?v=jS8cNtqKhyU (Accessed: 12 December 2024). cc/Class-5/Lab5.1 Introduction to Docker.md at master · warestack/cc (no date) GitHub. Available at: https://github.com/warestack/cc/blob/master/Class-5/Lab5.1%20Introduction%20to%20Docker.md (Accessed: 12 December 2024). cc/Class-6 at master · warestack/cc (no date) GitHub. Available at: https://github.com/warestack/cc/tree/master/Class-6 (Accessed: 12 December 2024). Continuous integration (100AD) Docker Documentation. Available at: https://docs.docker.com/build-cloud/ci/ (Accessed: 12 December 2024). Express Tutorial Part 3: Using a Database (with Mongoose) - Learn web development | MDN (2024). Available at: https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/mongoose (Accessed: 12 December 2024). Express Tutorial Part 4: Routes and controllers - Learn web development | MDN (2024). Available at: https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/routes (Accessed: 12 December 2024). Folder Structure for NodeJS & ExpressJS project (2022) DEV Community.

Available at: https://dev.to/mr_ali3n/folder-structure-for-nodejs-expressjs-project-435l (Accessed: 12 December 2024). joi.dev (no date) joi.dev. Available at: https://joi.dev/api/?v=17.13.3#stringemailoptions (Accessed: 12 December 2024). jsonwebtoken (2023) npm. Available at: https://www.npmjs.com/package/jsonwebtoken (Accessed: 12 December 2024). Kubernetes Deployment YAML File with Examples (no date) Spacelift. Available at: https://spacelift.io/blog/[slug] (Accessed: 12 December 2024). MongoDB Schema Design Best Practices (2021). Available at: https://www.youtube.com/watch?v=QAqK-R9HUhc (Accessed: 12 December 2024). Mongoose v8.8.4: SchemaTypes (no date). Available at: https://mongoosejs.com/docs/schematypes.html#objectids (Accessed: 12 December 2024). Node.js Authentication and Authorization with JWT: Building a Secure Web Application (2023) DEV Community. Available at: https://dev.to/taiwo17/nodejs-authentication-and-authorization-with-jwt-building-a-secure-web-application-236f (Accessed: 12 December 2024). Part 1: Containerize an application (100AD) Docker Documentation. Available at: https://docs.docker.com/get-started/workshop/02_our_app/ (Accessed: 12 December 2024). Secret Manager (no date) Google Cloud. Available at: https://cloud.google.com/security/products/secret-manager (Accessed: 12 December 2024). Service (no date a) Kubernetes. Available at: https://kubernetes.io/docs/concepts/services-networking/service/ (Accessed: 12 December 2024). Service (no date b) Kubernetes. Available at: https://kubernetes.io/docs/concepts/services-networking/service/ (Accessed: 12 December 2024).